

# SQL Data Analysis Report-E-Commerce Project

- By- UTTAM TIWARI

## 1. 🌐 Geographical Reach

**Business Question:** List all unique cities where customers are located.

**Logic:** Extract distinct locations to understand market spread.

CODE-

```
select distinct(customer_city) from customers;
```

## 2. 📅 Order Volume (2017)

**Business Question:** Count the number of orders placed in 2017.

**Logic:** Filter by year to analyze specific annual performance.

CODE-

```
select count(order_id) from orders where  
year(order_purchase_timestamp) = 2017;
```

## 3. 📦 Sales by Product Category

**Business Question:** Find the total sales per category.

**Logic:** Join products, orders & payments to aggregate revenue by category.

CODE-

```
select `product category`, round(sum(payments.payment_value),2) as  
total_sales  
from products  
join order_items  
on products.product_id = order_items.product_id  
join payments  
on order_items.order_id = payments.order_id  
group by `product category`  
order by total_sales desc;
```

## 4. 💳 Payment Behavior

**Business Question:** Calculate the percentage of order that were paid in installments.

**Logic:** Analyze payment table to find installment-based transactions.

CODE-

```
select round(sum(case when payments.payment_installments >= 1 then 1 else 0 end)/count(*)*100,3)as Installment_Percentage
from payments;
```

#### 5. 🧑 Customer Distribution by State

**Business Question:** Count number of customers from each state.

**Logic:** group by state to identify key markets.

CODE-

```
select customer_state as `Customer State`,count(customer_id) as
Customers from customers
group by customer_state
order by Customers desc;
```

#### 6. 📅 Monthly Order Trend (2018)

**Business Question:** Calculate the number of orders per month in 2018.

**Logic:** use date function to group data by month name

CODE-

```
select monthname(order_purchase_timestamp) as
Month_Name,count(order_id) as Order_Count
from orders
where year(order_purchase_timestamp) = 2018
group by Month_Name;
```

#### 7. 🛒 Average Basket Size

**Business Question:** Find the average number of products per order grouped by customer city.

**Logic:** Divide total items by total unique orders.

CODE-

```
select customers.customer_city,
round(count(order_items.product_id)/ count(distinct
orders.order_id),2) as Average_product
from customers
join orders
on customers.customer_id = orders.customer_id
```

```
join order_items
on orders.order_id = order_items.order_id
group by customers.customer_city
order by Average_product desc;
```

## 8. 💰 Revenue Contribution

**Business Question:** Calculate the percentage of total revenue contributed by each product category.

**Logic:** Use a subquery to calculate the grand total revenue for division.

CODE-

```
select `product category`, round(sum(payments.payment_value) /
(select sum(payments.payment_value) from payments)*100,2) as
Percentage_Contributed
from products
join order_items
on products.product_id = order_items.product_id
join payments
on order_items.order_id = payments.order_id
group by `product category`
order by Percentage_Contributed desc;
```

## 9. 🏆 Top Sellers

**Business Question:** Calculate the total revenue generated by each seller and rank them by revenue.

**Logic:** use dense rank() to handle ties in revenue ranking.

CODE-

```
select order_items.seller_id as Seller,
round(sum(order_items.price + order_items.freight_value ),2) as
Revenue,
dense_rank() over (order by round(sum(order_items.price +
order_items.freight_value ),2) desc ) as seller_rank
from order_items
group by order_items.seller_id
limit 5;
```

## 10. 📊 Moving Average

**Business Question:** Calculate the moving average of order value for each customer over their order history.

**Logic:** Advance window function using rows between unbounded preceding and current row

CODE-

```
select orders.customer_id,  
avg(payments.payment_value) over (partition by orders.customer_id  
order by orders.order_purchase_timestamp rows between unbounded  
preceding and current row)  
as Moving_Average  
from orders  
join payments  
on orders.order_id = payments.order_id  
where orders.order_status = "delivered"  
order by Moving_Average desc;
```

## 11. 📊 Cumulative Sales

**Business Question:** Calculate the cumulative sales per month for each year.

**Logic:** Use a CTE to aggregate monthly sales then apply a running total window function.

CODE-

```
with Monthly_Sales as (select year(  
orders.order_purchase_timestamp) as order_year,  
month(orders.order_purchase_timestamp) as order_month,  
round(sum(order_items.price + order_items.freight_value),2) as  
Revenue  
from orders  
join order_items  
on orders.order_id = order_items.order_id  
where orders.order_status = "delivered"  
group by year( orders.order_purchase_timestamp),  
month(orders.order_purchase_timestamp))  
select order_year, order_month, Revenue,  
sum(Revenue) over (partition by order_year order by order_month)  
as Cumulative_Sales  
from Monthly_Sales;
```

## 12. 🚀 Year-over-Year Growth

**Business Question:** Calculate the year-over-year growth rate.

**Logic:** Use LAG() to fetch previous years revenue and calculate percentage change.

CODE-

```
with Rev as (select year(orders.order_purchase_timestamp) as
order_year, round(sum(payments.payment_value),2) as Revenue,
lag(round(sum(payments.payment_value))) over ( order by
year(orders.order_purchase_timestamp)) as Pre
from orders
join payments
on orders.order_id = payments.order_id
group by order_year)
select order_year, Revenue,Pre,
from Rev;
(round((Revenue-Pre)/Pre*100)) as Growth_Percentage
```

### 13. Customer retention Rate

**Business Question:** Calculate the retention rate of customer defined as the percentage of customers who make another purchase within 6 month of their first purchase.

**Logic:** Complex Self-Join identifying first purchase vs subsequent purchase timestamp.

CODE-

```
with First_Order as (select customers.customer_unique_id,
min(orders.order_purchase_timestamp) as First_Purchase
from customers
join orders
on customers.customer_id = orders.customer_id
group by customer_unique_id),
Retained as (select distinct First_Order.customer_unique_id as
Retained_Customer
from First_Order
join customers
on First_Order.customer_unique_id = customers.customer_unique_id
join orders
on customers.customer_id = orders.customer_id
where orders.order_purchase_timestamp > First_Order.First_Purchase
and
orders.order_purchase_timestamp <=
date_add(First_Order.First_Purchase,interval 6 month))
select (round((select count(*) from Retained) / (select count(*)
from First_Order)*100,2))as Retention_Rate;
```

### 14. Top 3 Customer by Year

**Business Question:** Identify the top 3 customer who spent the most money in each year.

**Logic:** Partition ranking

CODE-

```
with TOP as (select year(orders.order_purchase_timestamp) as
Years, customers.customer_unique_id as Customers,
sum(payments.payment_value) as Spending,
dense_rank() over (partition by
year(orders.order_purchase_timestamp) order by
sum(payments.payment_value) desc) as Ranks
from customers
join orders
on customers.customer_id = orders.customer_id
join payments
on orders.order_id = payments.order_id
group by
year(orders.order_purchase_timestamp),customers.customer_unique_id
)
select * from TOP where Ranks <= 3;
```

#### 15. 🗺️ Top 3 Customer by State & Year

**Business Question:** Identify the state wise top 3 customer who spent the most money in each year.

**Logic:** Multilevel partition (year + state)

CODE-

```
with TOP as (select year(orders.order_purchase_timestamp) as
Years, customers.customer_unique_id as
Customers,customers.customer_state as State,
sum(payments.payment_value) as Spending,
dense_rank() over (partition by
year(orders.order_purchase_timestamp),customers.customer_state
order by sum(payments.payment_value) desc) as Ranks
from customers
join orders
on customers.customer_id = orders.customer_id
join payments
on orders.order_id = payments.order_id
group by
year(orders.order_purchase_timestamp),customers.customer_unique_id
,customers.customer_state)
select * from TOP where Ranks <= 3;
```